MECS 0033 Artificial Intelligence

Title: MAiCampus - Smart campus, student life management assistant

Course Code: MECS 0033
Course Name: Artificial Intelligence
Section: 52
Lecturer's Name: Dr. Shafaatunnur binti Hasan
Session / Semester: 25/26 (Sem 2)
Assignment Title: Assignment 3 (Intelligent Agent)

Group 2
Member Details

NO.	NAME	STUDENT ID
1	AISY MIKHAIL	MEC255055
2	DARWIN SUBRAMANIAM	MEC255043
3	TIAN ZHI FUNG	MEC255060
4	MOHAN MAHENDREN	MEC255068

Table of Contents

1.0 Objectives

The main objective of this project is to develop an intelligent and centralized University campus application with an assistant chatbot, which can assist UTM students and lecturers. This program would enhance how students manage their academic, administrative, and social activities within the university environment.

To achieve this, the project is guided by the following specific objectives:

Study Facilitation

- Manage class schedules, assignments, and project deadlines
- Provide access to research materials, books, and past-year papers

Area Facilitation

- Enable booking of campus facilities such as libraries, study rooms, and sports areas

Social Life Facilitation

- Provide a platform to explore and join university clubs
- Support club management, including admin roles, elections, and event updates

AI-Driven Assistance

- Provide intelligent chatbot support for student queries
- Deliver personalized recommendations and real-time guidance
- Assist with navigation and access to relevant contacts

2.0 Problem Statement

2.1 Introduction

University life rarely unfolds along a single track. Students are expected to juggle lectures, assignments, administrative paperwork, and a social calendar at the same time, and most of the information they need to do this is scattered across systems that were never really designed to talk to each other. Course outlines sit in one portal, grades in another, club announcements on Instagram, and room bookings on a form that only works properly on a desktop browser. The result is a daily friction that most students simply learn to live with.

This project proposes MAiCampus (an intelligent and centralized University campus application with an assistant chatbot) as an attempt to reduce that friction. Rather than adding yet another platform, the goal is to build a conversational layer that sits across the existing ones, so that a student can ask a single question and get a useful answer without chasing it through four different logins. The underlying idea is not particularly novel in the AI space, but its application to the specific conditions of a Malaysian public university, and to UTM in particular, has received little serious attention.

2.2 Problem Background

At UTM, as at many other universities, student information is spread thinly across a number of disconnected systems like the learning management system, faculty-specific portals, official email, WhatsApp groups maintained by class representatives, and the occasional physical notice board that still matters. Each of these serves a purpose, but together they produce an environment where relevant information is often technically available yet practically hard to find.

Four recurring issues emerge from this arrangement. The first is academic tracking. Students tend to rely on a patchwork of personal methods, screenshots, calendar apps, reminders from friends to keep up with deadlines, because no single institutional tool consolidates this view for them. The second is facility access. Booking a discussion room, a sports court, or a study pod typically requires knowing which system handles which resource, and the experience rarely feels consistent. The third is campus engagement. Clubs and student organisations are active, but information about their activities circulates mostly through informal channels, which tends to favour students who are already well-connected. The fourth is resource discovery. Past-year papers, reference materials, and research databases exist, but locating them often depends on knowing who to ask rather than where to look. On top of all this, there is no reliable way to get a quick answer to a routine administrative question outside office hours.

2.3 Significance of the problem

These issues may look like minor inconveniences individually, but their cumulative effect is more serious than it first appears.

Academically, missing a deadline because an announcement was buried in an email thread is not a trivial failure, such a situation will affect grades, and over time it affects confidence. Students who spend a significant portion of their week simply locating information have less time for the actual intellectual work their degree is supposed to cultivate. There is also an equity dimension here that is easy to overlook, for example students with stronger peer networks absorb institutional knowledge informally, while those without such networks transfer students, international students, or simply more introverted ones often fall behind not because of ability but because of access.

Engagement with campus life suffers in a similar way. When information about clubs and events is diffuse, participation skews toward those already in the loop, and the broader purpose of university life beyond the classroom is weakened. Facilities are also underused or contested unpredictably because the booking experience is opaque. Besides, there is a quieter cost in the form of cognitive load and low-grade frustration that accumulates across a semester.

2.4 Needs of an AI Solution

A conventional portal redesign could address some of these pain points, but it would not address the underlying problem, which is that students do not want to learn yet another interface. What they want is to ask a question in the way they would ask a friend and receive a useful answer. This is precisely the kind of task that recent advances in large language models and conversational AI have made tractable.

The proposed system is therefore built around a chatbot rather than a new dashboard, with the following core capabilities.

In terms of academic support, the assistant should be able to surface a student's schedule, upcoming deadlines, and outstanding tasks on request, and nudge them when priorities shift. It should also serve as an entry point to learning materials and past-year papers, so that discovery becomes a matter of asking rather than searching.

For facility management, the assistant should be able to check availability, initiate bookings, and recommend alternatives when a preferred slot is taken, without the student having to know which back-end system owns which resource.

On the social and community side, it should act as a directory and notice board for clubs and student organisations, lowering the barrier for students who are curious but not already embedded in a particular circle. Administrative features for club officers, such as managing membership or announcing events, can be layered on top of the same interface.

The conversational layer itself is what ties these capabilities together. Natural language interaction means the system is usable without training, available outside office hours, and increasingly personalised as it learns which questions a given student tends to ask. Finally, basic navigation and contact support such as finding a lecturer's office, locating a department, getting directions across campus,rounds out the set of everyday needs the assistant is meant to cover.

None of these features are revolutionary in isolation. What the project proposes is that bringing them together behind a single conversational interface, tuned to the specific conditions of UTM, can produce a meaningfully better student experience than the current fragmented arrangement allows.

3.0 Empathy Map

- Says
    - "Class venue is far, buses are not punctual."
    - "Campus is too big, a long walk to the hall."
    - "Hard to locate the lecturer's office desk."
    - "Too many assignments, no time."
    - "Want one app for all the information."
    - "Make visa application optional."

- Thinks
    - "I'll forget another deadline."
    - "Is there a free slot today or not?"
    - "Am I missing an event everyone knows?"
    - "Where do I even find past year papers?"
    - "Would AI be faster than asking a lecturer?"
    - "Did I submit the right visa document?"

- Does
    - Relies on phone calendar, to-do lists, memory.
    - Check LMS and portal for deadlines.
    - Asks friends or uses trial and error to navigate.
    - Googles past year papers, scrolls social media.
    - Hears about events via posters, WhatsApp, word.
    - Already uses AI chatbots for university tasks.

- Feels
    - Overwhelmed by too many deadlines.
    - Anxious about forgetting or missing out.
    - Frustrated by confusing booking systems.
    - Excluded from club and campus events.
    - Confused by visa and admin paperwork.
    - Needs an all-in-one solution.

- Pains
    - Forgetting deadlines, poor communication.
    - Group work friction and coordination issues.
    - Facility booking: confusing, no slots, slow.
    - Clubs are hard to discover, finding out too late.
    - Study materials scattered across the portal, Google.
    - Lost on campus, rooms relocated last minute.
    - Visa: document prep, requirements, waiting.
    - Parking, air quality, lecture schedule changes.
    - Time management with too much on their plate.
    - Library and portal updates feel delayed.

- Gains
    - One place for schedule, deadlines, materials.
    - Assignment tracker with smart reminders.
    - Calendar sync and push notifications.
    - Easy facility booking with live availability.
    - Club directory, event feed, membership signup.
    - Chat and community features for peers.
    - AI chatbot for study and admin questions.
    - Clearer visa and document guidance.
    - Campus map or navigation that actually works.
    - File storage and collaboration for group work.

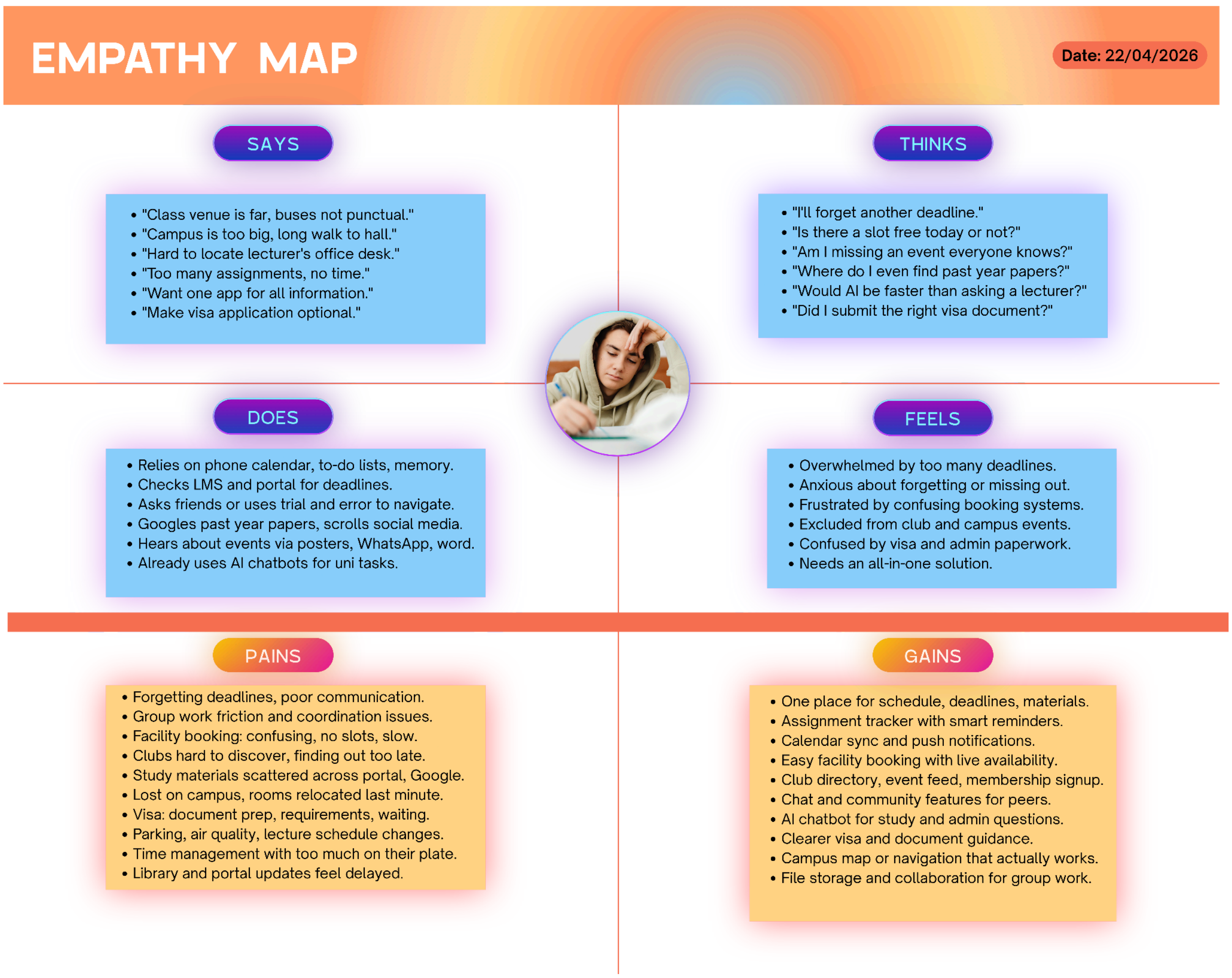

Figure 3.1 MAiCampus Empathy Map

4.0 Architecture

MAI Campus is a cross-platform AI student companion built on the Flet framework (Python UI over Flutter). The proposed architecture follows a modular, event-driven client pattern in which the application is a single long-running process that composes four cooperating subsystems: the Front-End UI, the Chatbot runtime, the Notification pipeline, and the Knowledge Repository. The client is designed to function in an offline-capable, privacy-first posture.All user data is persisted locally (TinyDB for structured records, ChromaDB for vector embeddings) and only leaves the device when the chatbot explicitly calls a configured Large Language Model (LLM) provider.

At the highest level the design separates three concerns:

1. **Presentation** — a reactive UI tree rendered by Flet, driven by user navigation and asynchronous state updates.
2. **Intelligence**— a provider-agnostic chatbot runtime that streams tokens, invokes registered tools, and retrieves relevant memories as context.
3. **Persistence & Knowledge** — two complementary repositories (episodic vector memory and a structured campus knowledge store) that together form the retrieval substrate for the chatbot.

A background **Planner** task periodically scans the structured store and emits alerts through the Notification pipeline, closing the loop between state changes in the knowledge repository and proactive communication with the student.

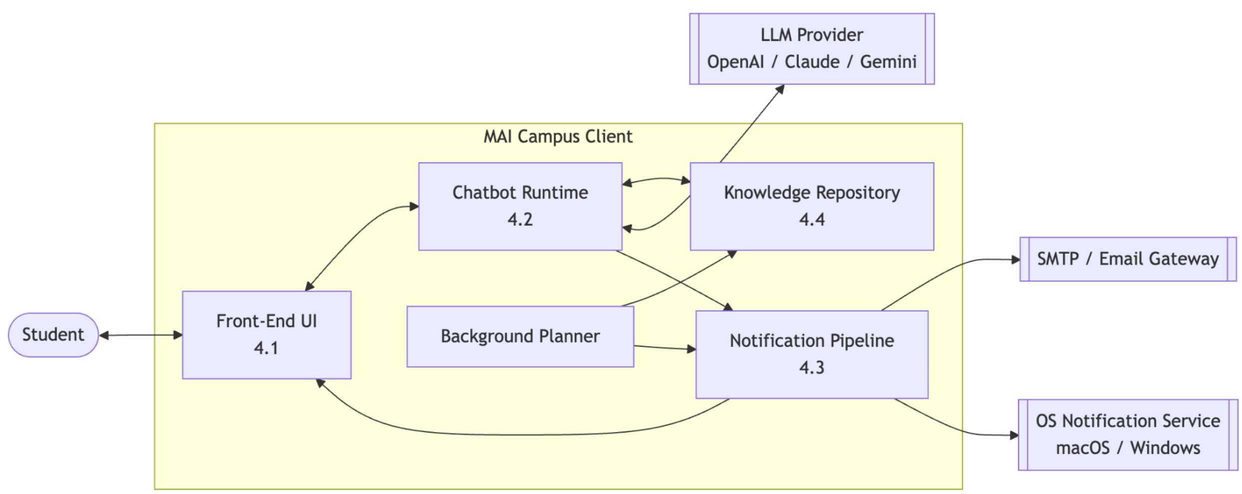

Figure 4.1: Overview Architecture Diagram (Proposed)

Table 4.1: System Components and Responsibilities

Subsystem	Component	Description	Key Technologies
Front-End UI	Flet UI & State Manager	Handles user interface rendering, interaction, and state updates	Flet (Python over Flutter)
Front-End UI	Navigation & Chat UI	Manages screen routing and provides chatbot interaction interface	Flet Widgets
Chatbot Runtime	LLM Interface & Context	Processes queries, retrieves context, and generates responses	OpenAI API / Local LLM
Chatbot Runtime	Tool Invocation Engine	Executes functions such as booking, search, and navigation	Python Functions / APIs
Notification Pipeline	Planner & Alerts	Monitors data and generates real-time notifications	Background Scheduler
Knowledge Repository	TinyDB & ChromaDB	Stores structured data and vector embeddings for retrieval	TinyDB, ChromaDB

5.0 Survey Summary

A Google Form survey was distributed to UTM students to identify the most frequent pain points in campus life. A total of 21 responses were collected across multiple faculties (Business 33%, Engineering 24%, IT/CS 19%) and year groups (Year 4+ 57%, Year 1–3 43%). The findings directly shaped MAiCampus feature priorities and UI design.

5.0.1 Academic Task Management

Table 5.1 — Academic Task Management

Question	Key Finding
How do you manage academic tasks?	33% to-do lists; 29% phone apps; 29% have no systematic method at all.
How do you track assignments and deadlines?	43% phone calendar, 33% rely on memory alone, only 10% use the LMS.
Most frustrating part of assignments?	Poor communication, too many deadlines, forgetting, group work friction (cited across >80% of responses in various combinations).
If everything was in ONE place, how useful? (1–5)	57% rated 5/5 and 38% rated 4/5 and 95% gave a score of 4 or 5.
Most valued features?	Assignment tracker, notifications/reminders, calendar integration, AI support.

***Insight**: Many students lack a systematic approach. Memory and disconnected phone apps dominate, and 95% of respondents see strong value in a unified platform.*

5.0.2 Facility Booking

Table 5.2 — Facility Booking

Question	Key Finding
How often do you use university facilities?	43% daily, 38% occasionally high and consistent usage.
Problems when booking facilities?	24% cite no available slots, confusing system and 'don't know how to book' also frequently reported.
What matters most when booking?	Ease of booking, availability, and time slots ranked highest.

***Insight**: Facility use is frequent, but the booking experience is fragmented. Students cannot easily check availability or navigate the booking system.*

5.0.3 Club and Campus Engagement

Table 5.3 — Club and Campus Engagement

Question	Key Finding
How easy is it to discover clubs? (1 = easy, 5 = hard)	52% rated 3 (moderate) only 5% rated 5. Hence, majority find it moderately hard.
Would you use an app for club browsing?	48% Yes, 33% Maybe. Hence, 81% open to using one.
Ever felt excluded from campus events?	33% Yes, 52% Sometimes. Hence, 85% felt out of the loop at least occasionally.

***Insight**: Club discovery relies on informal channels (WhatsApp, word-of-mouth), disadvantaging students not already embedded in social networks.*

5.0.4 Study Materials and Resources

Table 5.4 - Study Materials and Resources

Question	Key Finding
How difficult is it to find textbooks / past papers? (1–5)	29% rated 4 (difficult), 10% rated 5 (very difficult). Hence, 39% find it difficult to very difficult.

***Insight**: Materials are technically available but practically hard to locate. Students depend on knowing who to ask, not where to look.*

5.0.5 AI and Chatbot Readiness

Table 5.5 — AI and Chatbot Readiness

Question	Key Finding
Would you use an AI chatbot for study support?	81% Yes, 19% Maybe. Hence, 100% willing
Have you already used an AI chatbot for university tasks?	81% already have. Hence AI adoption is already high.
Would you use an AI reminder system?	67% Yes, 14% Maybe. Hence, 81% open to it.
Prefer direct professor contact or AI middleman?	43% direct, 38% both, 19% prefer AI.

Insight: *There is zero resistance to the chatbot concept. Most respondents already use AI tools for university work. Therefore, we can say, MAiCampus is a natural evolution, not a new habit.*

5.0.6 Top Daily Hassles (Open Response)

When asked for the single biggest hassle in daily university life, students reported: time management and too many assignments, campus navigation (relocated classes, cannot find rooms), bus and transport delays, no fixed lecture schedule, and delays in getting academic updates. These responses directly informed the Smart Planner, Calendar Sync, and Campus Navigation features of MAiCampus.

5.0.7 Key Statistics Summary

Table 5.6 — Key Statistics Summary

Statistic	Value
Respondents who want everything in one platform (rated 4–5/5)	95%
Respondents already using AI chatbots for university tasks	81%
Respondents willing to use an AI chatbot for study support	100%
Respondents who felt excluded from campus events	85%
Respondents who track deadlines using memory alone	33%
Respondents open to AI reminder system	81%
Respondents citing facility booking as a pain point	>70%

5.1 User Interface Storyboard

The User Interface Storyboard for MAiCampus was designed following the Define phase of the Design Thinking process. Survey responses and stakeholder insights (see Section 5.0 Appendix) were used to identify the most critical student pain points, which then shaped the screen flow and navigation architecture. The full UI navigation flow diagram is available at: https://canva.link/tbmnu0iiqsmw4o1

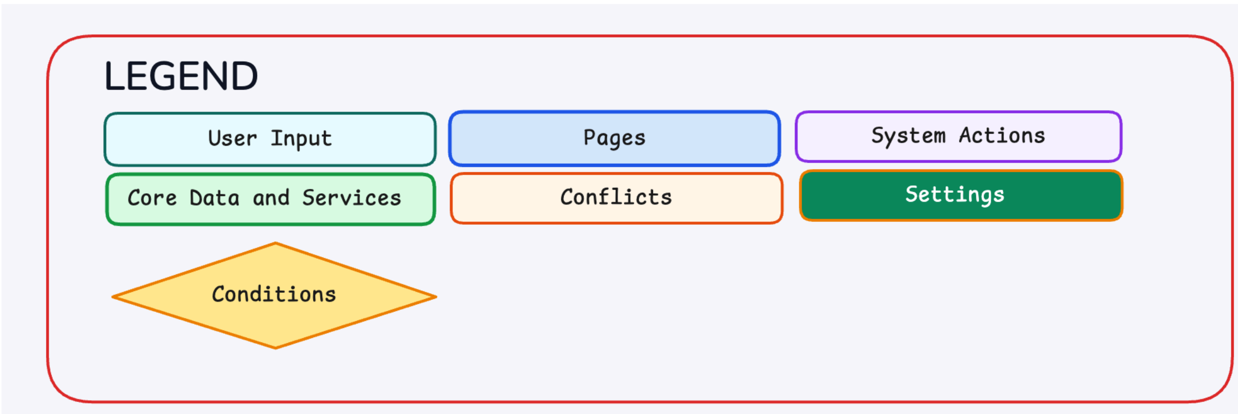

Figure 5.1 Legend for UI Flow Diagram

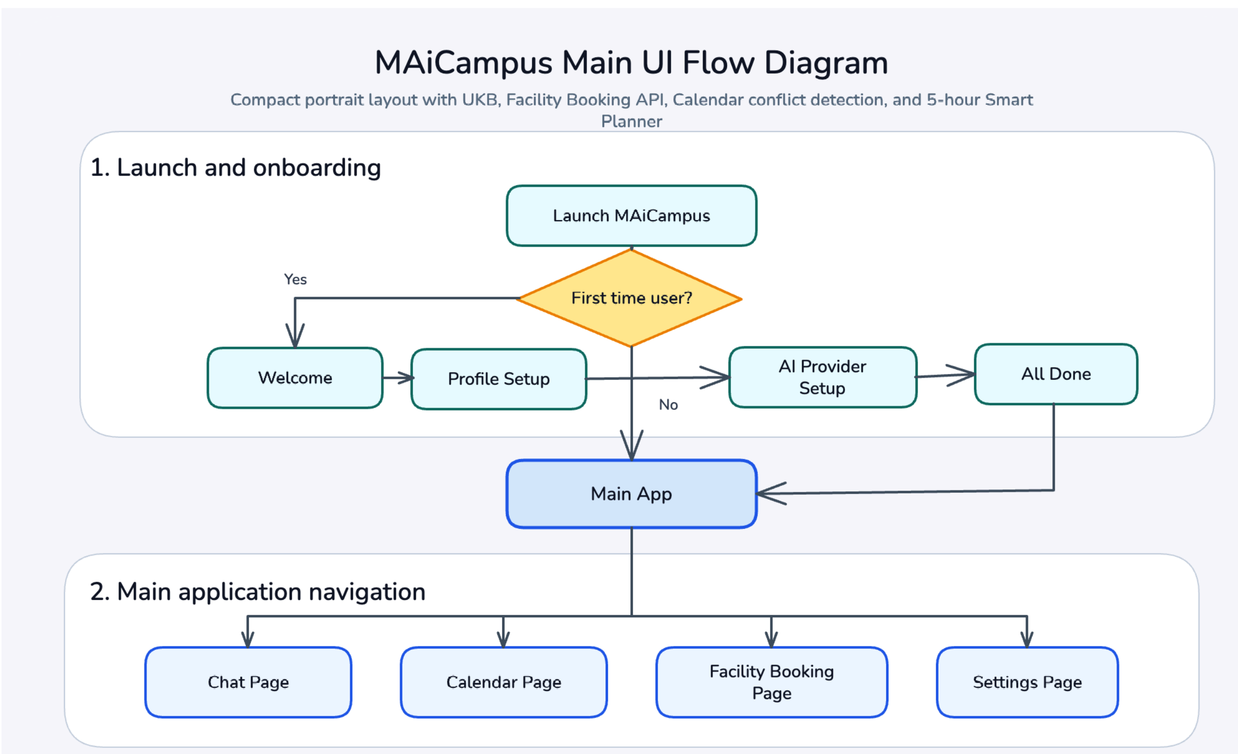

Figure 5.2 UI Flow Diagram Part 1

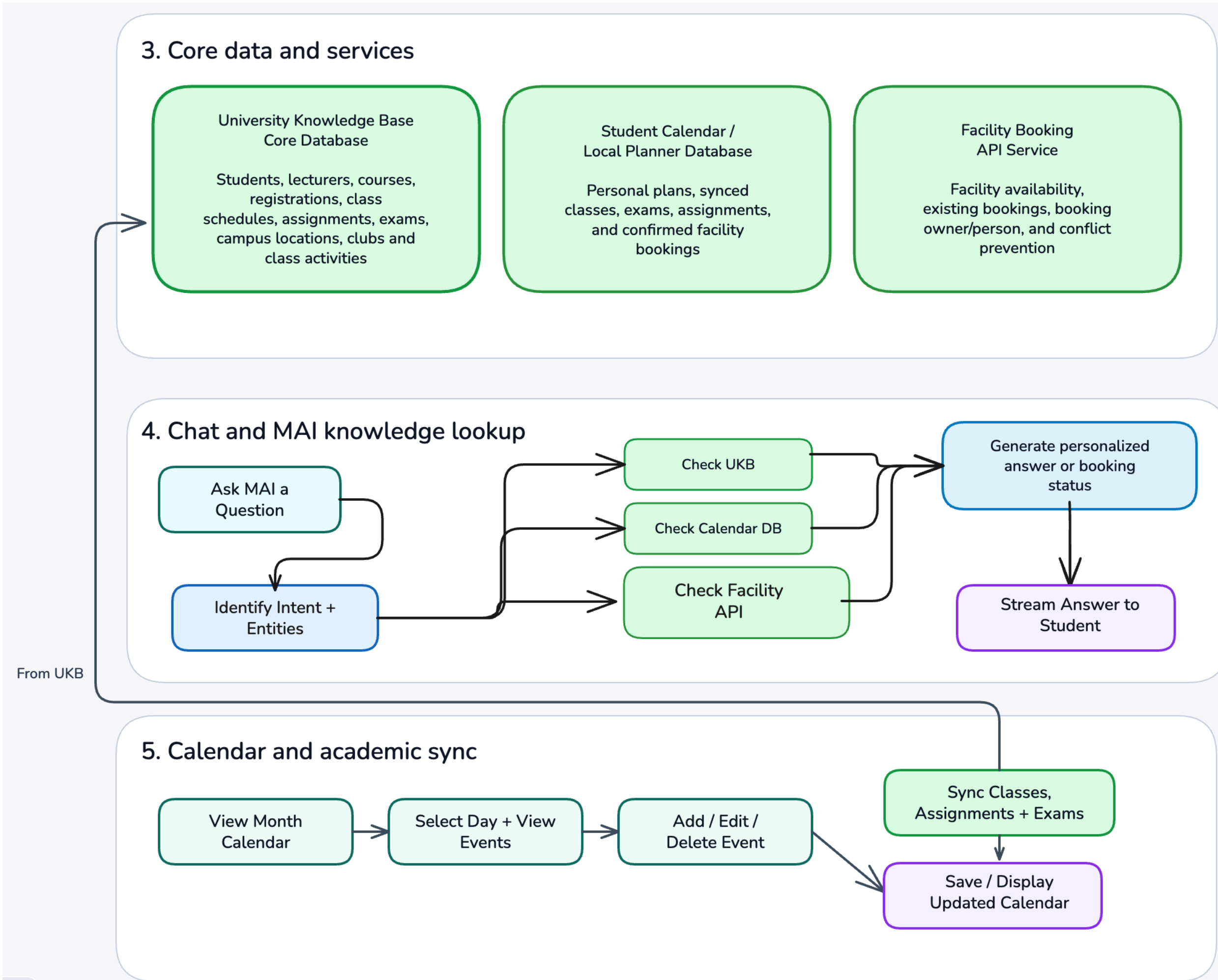

Figure 5.3 UI Flow Diagram Part 2

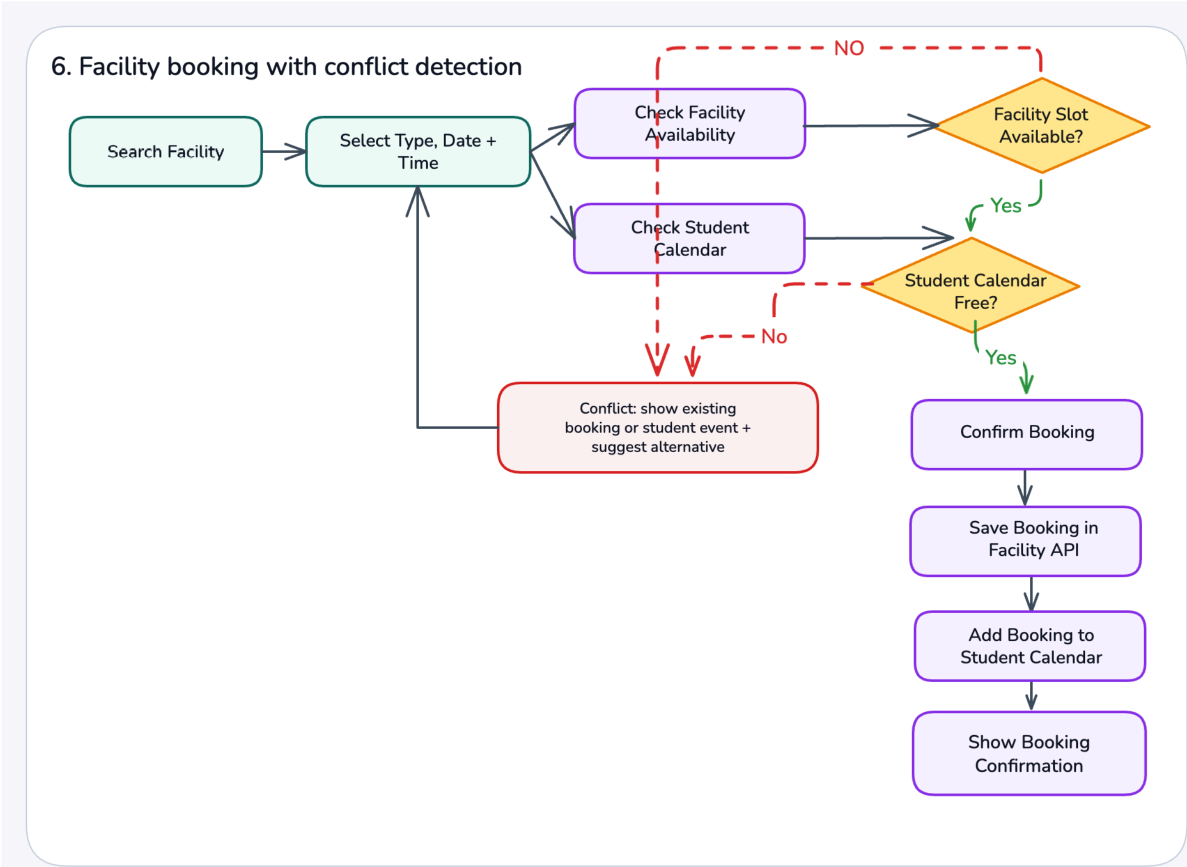

Figure 5.4 UI Flow Diagram Part 3

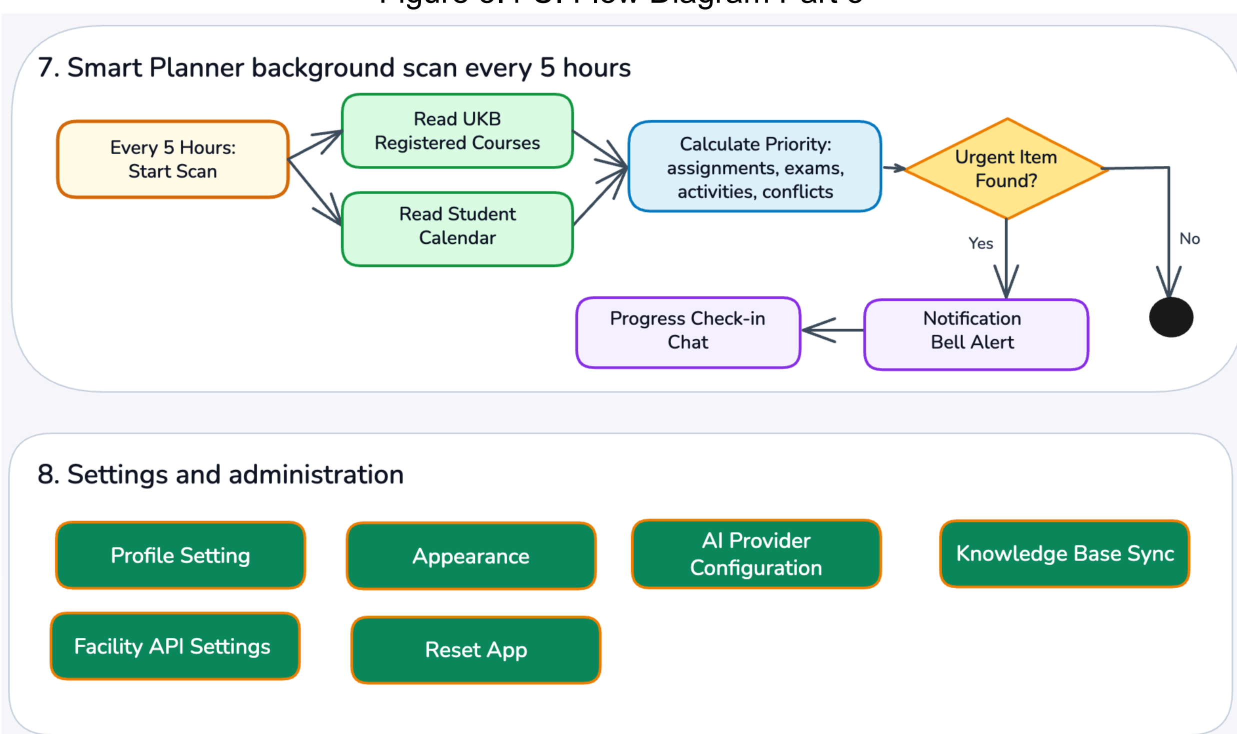

Figure 5.5 UI Flow Diagram Part 4

5.2 UI Navigation Flow

The MAiCampus UI is structured into eight functional flows. Each flow maps a user journey through screens and decision points. Table 4.2 summarizes all screens and their purpose.

Table 5.7 — MAiCampus UI Navigation Flow Summary

Flow	Screen / Step	Purpose
1. Launch & Onboarding	Launch MAiCampus	App entry point.
	First Time User? (condition)	Route new vs returning users.
	Welcome	Greeting and orientation.
	Profile Setup	Enter student name and ID
	AI Provider Setup	Configure preferred LLM provider.
	All Done → Main App	Complete onboarding, enter the app.
2. Main Navigation	Chat Page	Converse with MAI chatbot
	Calendar Page	View and manage academic schedule.
	Facility Booking Page	Search and book campus facilities.
	Settings Page	Manage profile, AI, planner, sync.
3. Core Data & Services	University Knowledge Base (UKB)	Students, courses, schedules, exams, clubs.
	Student Calendar / Planner DB	Personal plans, synced classes, bookings.
	Facility Booking API	Facility availability and conflict prevention.
4. Chat & MAI Lookup	Ask MAI a Question	User inputs natural language query.
	Identify Intent + Entities	NLP parses intent and key entities.
	Check UKB / Calendar / Facility	Retrieve relevant data from services.
	Generate Personalised Answer	LLM synthesises responses.
	Stream Answer to Student	Deliver answers via chat interface.
5. Calendar Sync	View Month Calendar	Monthly overview of all events.
	Select Day + View Events	Drill down to a daily schedule.

	Sync Classes / Assignments / Exams	Pull scheduled items from UKB.
	Add / Edit / Delete Event	Manual calendar management.
6. Facility Booking	Search Facility	Filter by type, location.
	Select Type, Date + Time	Choose booking parameters.
	Check Facility Availability	Query Facility Booking API.
	Check Student Calendar	Detecting personal calendar conflicts.
	Facility / Calendar Free? (Y/N)	Decision: proceed or suggest alternatives.
	Confirm Booking → Save → Notify	End-to-end booking confirmation.
7. Smart Planner	Every 5 Hours: Start Scan	Background priority scan.
	Read UKB + Student Calendar	Gather upcoming items.
	Calculate Priority	Rank assignments, exams, conflicts.
	Urgent Item Found? (Y/N)	Trigger notification or check-in chat.
8. Settings	Profile / Appearance / AI / Sync	Manage all app configuration.

6.0 Knowledge Base — Knowledge Representation in Predicate Logic

The Knowledge Base (KB) of MAiCampus stores factual knowledge about the university domain. The two primary entities modelled in this phase are Course and Course Timetable, as required by the assignment specification. A Course represents any academic subject registered at UTM. A course Timetable records a scheduled occurrence of a course, capturing the day, start time, end time, room, and lecturer.

6.1 Predicate Symbol Definitions

The following predicate symbols are used in the KB:

Table 6.1 — Predicate Symbol Definitions

Predicate Symbol	Meaning
Course(c)	c is a course offered at UTM
CourseTimeTable(ct)	ct is a scheduled timetable entry
Student(s)	s is a registered student
EnrolledIn(s, c)	student s is enrolled in course c
HasSchedule(c, ct)	course c has timetable entry ct
OnDay(ct, d)	timetable entry ct is on day d
StartTime(ct, t)	timetable entry ct starts at time t
EndTime(ct, t)	timetable entry ct ends at time t
InRoom(ct, r)	timetable entry ct is held in room r
TaughtBy(c, l)	course c is taught by lecturer l
HasClass(s, ct)	student s has a class at timetable entry ct
AskSchedule(s)	student s queries MAI about their schedule
CanRespond(bot, s, ct)	chatbot can reply to student s about timetable ct
CourseOnDay(c, d)	course c has at least one class on day d

6.2 Knowledge Representations

Table 6.2 presents 12 knowledge representations grounded in the MAiCampus domain. Facts (rows 1–9) are ground atoms using specific constants. Rules (rows 10–12) use universal quantification to capture general domain knowledge.

Table 6.2 — Knowledge Representations in FOL for MAiCampus KB

No.	English Sentence	FOL Representation
1	MECS0033 is a course at UTM.	Course(MECS0033)
2	AI_Mon_9am is a timetable entry.	CourseTimeTable(AI_Mon_9am)

3	Darwin is enrolled in course MECS0033.	EnrolledIn(darwin, MECS0033)
4	Course MECS0033 is scheduled in timetable entry AI_Mon_9am.	HasSchedule(MECS0033, AI_Mon_9am)
5	Timetable entry AI_Mon_9am is on Monday.	OnDay(AI_Mon_9am, Monday)
6	Timetable entry AI_Mon_9am starts at 9:00 AM.	StartTime(AI_Mon_9am, 9am)
7	Timetable entry AI_Mon_9am ends at 11:00 AM.	EndTime(AI_Mon_9am, 11am)
8	Timetable entry AI_Mon_9am is held in room N28.	InRoom(AI_Mon_9am, N28)
9	Course MECS0033 is taught by Dr. Shafaatunnur.	TaughtBy(MECS0033, Dr_Shafaatunnur)
10	If a student is enrolled in a course and that course has a timetable entry, then the student has a class at that entry.	∀x, y, z (EnrolledIn(x, y) ∧ HasSchedule(y, z) → HasClass(x, z))
11	If a student queries MAI about their schedule and is enrolled in a course that has a timetable entry, then MAI can respond.	∀x, y, z (AskSchedule(x) ∧ EnrolledIn(x, y) ∧ HasSchedule(y, z) → CanRespond(chatbot, x, z))
12	If a course has a timetable entry scheduled on day d, then the course has a class on day d.	∀x, y, d (HasSchedule(x, y) ∧ OnDay(y, d) → CourseOnDay(x, d))

These 12 representations collectively allow MAI to answer queries such as 'What class do I have on Monday?', 'Who teaches my AI course?', and 'When does my MECS0033 class end?' by querying the KB using inference.

7.0 State Space Graph — Resolution Refutation

Resolution refutation is a proof technique used to determine whether a goal G follows from the KB. The method works by negating the goal (adding ¬G to the KB) and applying the resolution rule repeatedly until the empty clause NIL is derived. Deriving NIL confirms that G is true.

7.1 Scenario

Scenario: A student (Darwin) opens MAiCampus and asks MAI:

> *"Do I have any class on Monday?"*

MAI must prove from the KB that Darwin has a class at timetable entry AI_Mon_9am.

Goal (G): HasClass(darwin, AI_Mon_9am)
Negated Goal (¬G): ¬HasClass(darwin, AI_Mon_9am)

7.2 Premises Selected from KB

The following premises from Table 7.2 are relevant to this proof:

Table 7.1 — KB Premises Used in the Proof

Label	Premise (Natural Language)	FOL
P1	Darwin is enrolled in MECS0033.	EnrolledIn(darwin, MECS0033)
P2	MECS0033 has a timetable entry AI_Mon_9am.	HasSchedule(MECS0033, AI_Mon_9am)
P3	If a student is enrolled in a course that has a timetable entry, the student has a class there.	∀x,y,z (EnrolledIn(x,y) ∧ HasSchedule(y,z) → HasClass(x,z))

7.3 CNF Conversion

Before resolution can be applied, all premises and the negated goal must be expressed in Conjunctive Normal Form (CNF). Implications are eliminated using A → B ≡ ¬A ∨ B, and universal quantifiers are dropped (Skolemisation not required as there are no existential quantifiers).

Table 7.2 — FOL to CNF Conversion

Clause	Original FOL	CNF Form	Notes
C1	EnrolledIn(darwin, MECS0033)	EnrolledIn(darwin, MECS0033)	Ground atom — already in CNF
C2	HasSchedule(MECS0033, AI_Mon_9am)	HasSchedule(MECS0033, AI_Mon_9am)	Ground atom — already in CNF
C3	∀x,y,z (EnrolledIn(x,y) ∧ HasSchedule(y,z) → HasClass(x,z))	¬EnrolledIn(x,y) ∨ ¬HasSchedule(y,z) ∨ HasClass(x,z)	Implication eliminated; ∀ dropped
C4	¬HasClass(darwin, AI_Mon_9am)	¬HasClass(darwin, AI_Mon_9am)	Negated goal added for refutation

7.4 Resolution Steps

Table 7.3 — Resolution Steps

Step	Clauses Resolved	Substitution θ	Resolvent
1	C3 and C4	{x ← darwin, z ← AI_Mon_9am}	¬EnrolledIn(darwin, y) ∨ ¬HasSchedule(y, AI_Mon_9am)
2	Result of Step 1 and C1	{y ← MECS0033}	¬HasSchedule(MECS0033, AI_Mon_9am)
3	Result of Step 2 and C2	{ }	NIL ✔

7.5 Proving Tree

Figure 7.1 shows the proving tree. Each node is a clause; edges represent a resolution step with the substitution applied. NIL at the leaf confirms the goal is proved.

```
C3: ¬EnrolledIn(x,y) ∨ ¬HasSchedule(y,z) ∨ HasClass(x,z)
                                        C4: ¬HasClass(darwin, AI_Mon_9am)
                                       /
 Step 1  {x←darwin, z←AI_Mon_9am}
 ¬EnrolledIn(darwin,y) ∨ ¬HasSchedule(y, AI_Mon_9am)
                   C1: EnrolledIn(darwin, MECS0033)
                  /
 Step 2  {y←MECS0033}
 ¬HasSchedule(MECS0033, AI_Mon_9am)
                   C2: HasSchedule(MECS0033, AI_Mon_9am)
                  /
 Step 3  { }
 NIL ✓
```

Figure 7.1 — Proving Tree for HasClass(darwin, AI_Mon_9am)

7.6 State Space Graph — Resolution Refutation Summary

Since NIL is derived after three resolution steps, it is proven by refutation that ***HasClass(darwin, AI_Mon_9am)*** is **TRUE**. MAI can therefore correctly respond to Darwin:

> *"Yes, you have MECS0033 (Artificial Intelligence) on Monday from 9:00 AM to 11:00 AM in Room N28, taught by Dr. Shafaatunnur."*

This knowledge was retrieved entirely from the KB using automated theorem proving, demonstrating the practical value of FOL-based knowledge representation for MAiCampus.

8.0 PEAS Model Diagram

Figure 8.1 shows the agent-environment loop for MAiCampus. The environment (left) emits percepts that the agent reads through its sensors (top). The performance measure (right) continuously evaluates how good the agent's behaviour is. Then, an action is carried out by the actuators (bottom), which in turn changes the environment.

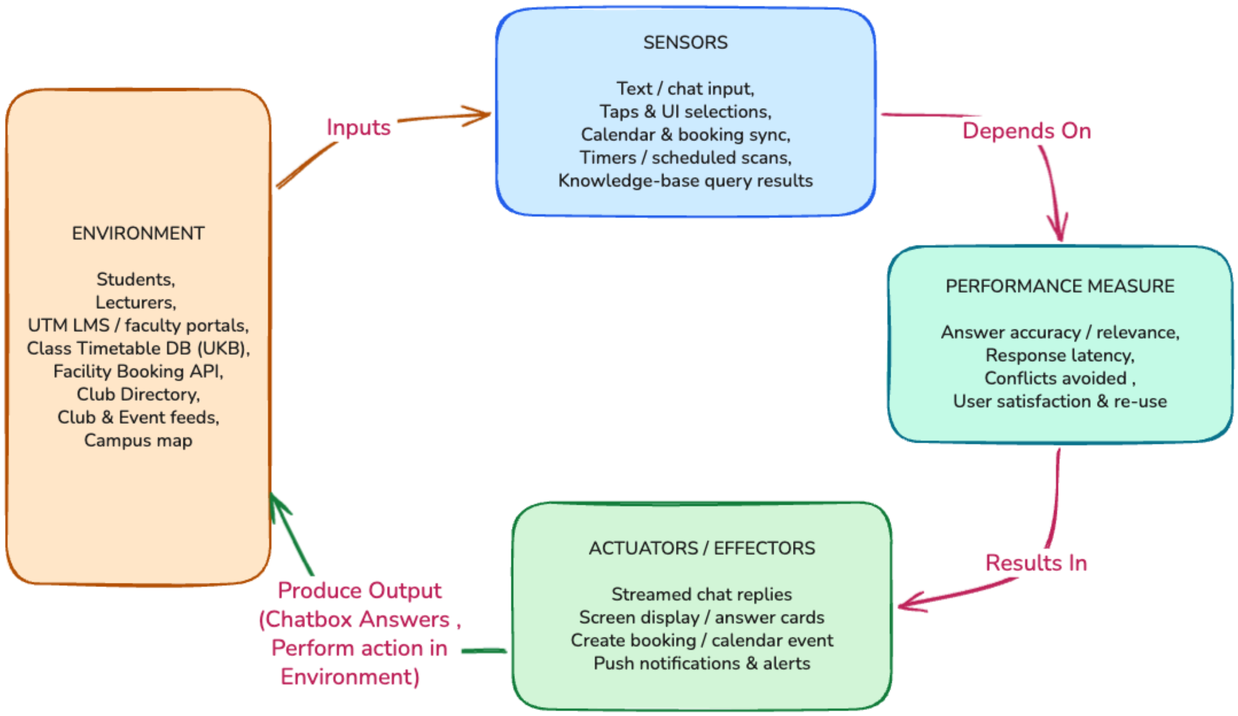

Figure 8.1 — PEAS model and agent-environment loop for MAiCampus

8.1 Detailed Definition of Each PEAS Property

8.1.1 P — Performance Measure

The performance measure defines what it means for MAiCampus to behave well. Because the goal of the project is to reduce friction in student life, the measure rewards correct, fast, complete and trusted assistance rather than mere availability. Concretely, MAiCampus is evaluated on:

- **Answer accuracy / relevance:** the proportion of answers that are factually correct and relevant to the question asked (drawn from the verified campus stores, not hallucinated).
- **Response latency:** time from a student's message to a useful streamed reply; the chat flow streams the answer to keep perceived latency low.

- **Conflicts avoided:** deadlines surfaced on time and booking/calendar conflicts avoided, reflecting the Smart Planner's background scan.
- **User satisfaction & re-use:** repeat usage and satisfaction ratings, indicating the assistant is trusted enough to replace the current patchwork of tools.

POC representation: These metrics are observable in the A2 UI: *accuracy and satisfaction* via thumbs-up/down on streamed answers, completion via successful booking/calendar confirmations, *latency* via response timing, and planner effectiveness via the priority alerts in flow 7 (Figure 5.4 Smart Planner).

8.1.2 E — Environment

The environment is everything outside the agent that it senses and acts upon. For MAiCampus this is the UTM digital and human ecosystem:

- **Human users:** students and lecturers who converse with the assistant in natural language (the public users in the storyboard).
- **Campus data sources:** the LMS and faculty portals, the Class Timetable / University Knowledge Base (UKB) store, the Facility Booking API, club directory, club and event feeds, and the campus map.
- **Knowledge repository:** the dual-store knowledge repository which is made of semantic memory (Mem0 + ChromaDB) for fuzzy, embedding-searchable history, and a structured campus store for explicit, queryable schedule and enrollment facts.

POC representation: The A2 architecture's data layer (UKB Core DB, Student Calendar DB, Facility Booking API) and the onboarding 'AI Provider Setup' screen together define this environment.

8.1.3 A — Actuators / Effectors

Actuators are the means by which the agent acts on the environment. The effectors are software outputs that change what the user sees and the state of campus systems:

- **Streamed chat replies:** natural-language answers streamed token-by-token into the chat view (A2 flow 4: Figure 5.3 , 'Stream Answer to Student').
- **Screen display / answer cards:** rendered cards, schedules and calendar/booking views shown on screen.
- **Booking & calendar actions:** write-actions that create a facility booking or add a class/assignment to the student calendar (A2 flows 5:Figure 5.3) and (A2 flows 6:Figure 5.4).
- **Push notifications & alerts:** priority alerts and reminders emitted by the Smart Planner (A2 flow 7: Figure 5.5).

POC representation: Each actuator corresponds to an output element of the UI -- the chat stream, calendar screen, facility booking confirmation and the notification system described in Figure 4.1: section 4.3 of the architecture.

8.1.4 S — Sensors

Sensors are how the agent perceives the environment. MAiCampus reads both the user's request and the live state of campus data:

- **Text / chat input:** typed messages captured from the chat box (A2 flow 4: Figure 5.3, 'Ask').
- **Taps & UI selections:** menu taps, day selection, facility search and settings choices across the UI.
- **Calendar & booking sync:** the current contents of the timetable, calendar and booking systems read on demand.
- **Timers / scheduled scans:** the clock and the Smart Planner's background scan (every ~5 hours) that wakes the agent to re-evaluate priorities.
- **Knowledge-base query results:** facts and embeddings returned from the dual-store knowledge base when the agent queries it.

POC representation: Sensors map to the input side of the A2 chat flow where, the NLP stage that 'Identifies Intent + Entities' from the captured text, plus the sync calls that read the UKB, calendar and booking API before a response is synthesised.

8.2 PEAS Properties Summary Table

Table 8.1 summarises the four PEAS properties of the MAiCampus intelligent agent.

PEAS Property	Description for MAiCampus
Performance Measure	Answer accuracy & relevance, response latency, conflict avoided, user satisfaction and repeat usage.
Environment	UTM students and lecturers, LMS & faculty portals, Class Timetable / UKB store, Facility Booking API, club directory, club & event feeds, campus map, the dual-store knowledge repository (semantic + structured).
Actuators / Effectors	Streamed natural-language chat replies, on-screen cards and calendar/booking views, create-booking and add-to-calendar write actions, push notifications and priority alerts.
Sensors	Text/chat input, UI taps and selections, calendar & booking sync reads clock and background scan timers, knowledge-base query results.

Table 8.1 — PEAS model properties for the MAiCampus intelligent agent

9.0 Appendix(Survey Form and Responses)

Mind Map Link (Canva):
https://canva.link/tbmnu0iiqsmw4o1

Google Form Link:

Survey Form (Google Forms):
https://docs.google.com/forms/d/e/1FAIpQLSdQbNBgOlr6vmDNKaqVsFZXViErGgqBjOAqPUoHHqgm52ep5Q/viewform?usp=dialog

Survey Responses (Google Sheets)
https://docs.google.com/spreadsheets/d/1JhdbMCzuKh261Lv1AoMKfvEHSyWmZB5k1ptEGV9WH5w/edit?usp=sharing